

Overview

Part 0 —  Launchpad · 00—launchpad/README.md

00 — Launchpad

Read this before Module 01. Fifteen minutes now saves you re-reading module intros later.

How this track is built

Every numbered module (01–16) follows the same shape:

```
<module>/
├── README.md           the syllabus: objectives, suggested path, exit check
├── notes/              01-core-concepts.md (teaching prose) + 02-deep-dive.md (craft)
├── code/               2+ runnable, heavily commented scripts – no API keys required
├── lab/EXERCISES.md    graded exercises with stated deliverables and checks
├── papers/PAPERS.md    primary sources – real papers, real URLs, annotated
├── resources/RESOURCES.md courses, repos, communities
└── slides/             deck.json (source) + a built .pptx + a Marp-compatible slides.md
```

Open a module's `README.md` first. It tells you what order to read the rest in and roughly how long each piece takes. The "exit check" at the bottom of every README is the actual bar — if you can do it without looking things up, move on.

Setup

Almost everything runs on plain Python + NumPy. A few modules use PyTorch, scikit-learn, or a specific package named in that module's `code/requirements.txt`.

```
# from the repo root
python3 -m venv .venv
.venv/bin/pip install --upgrade pip
.venv/bin/pip install numpy scikit-learn          # covers most modules
# then, per module, if it has one:
.venv/bin/pip install -r <module>/code/requirements.txt
```

No script in this repository requires an API key or network access to run. Anything that talks to a real hosted service (Bedrock, LangSmith, the Claude API) is either simulated with a deterministic fake, or guarded to print a clear explanation and exit cleanly when credentials aren't present.

Rebuilding the slide decks

Every deck is authored as `slides/deck.json` (see `_tools/AUTHORING_SPEC.md` for the schema) and compiled to PowerPoint + Markdown:

```
.venv/bin/python _tools/build_decks.py          # rebuild every deck in the repo
.venv/bin/python _tools/build_decks.py 04-llm    # rebuild just one module
.venv/bin/python _tools/build_decks.py claude-code # matches by folder name anywhere, incl.
tracks/
```

The PDF book

The whole track also compiles into one print-ready PDF — every module's notes, labs, code templates and paper lists, with a table of contents, clickable outline and continuous page numbers:

```
.venv/bin/pip install markdown pypdf websockets # one-time
.venv/bin/python _tools/build_book.py           # -> book/AI-End-to-End-Learning-Track.pdf
.venv/bin/python _tools/build_book.py 04-llm 08-rag # or just some parts, for a quick look
```

Each chapter is also written out as its own PDF under `book/chapters/`, numbered in book order, so you can hand out (or print) a single module without the other 600 pages.

The suggested order

1–4 are load-bearing — do not skip them to get to LLMs faster. 5–11 are applied AI, roughly independent of each other but each assumes 1–4. 12–15 (governance/security/compliance/evals) are the responsible-AI layer and read best after you've built something in 5–11. 16 is the hands-on capstone: seven tracks covering the languages and frameworks you'll actually write code in.

Full map and search: open `../README.html` in a browser.

If you only have one week

Modules 01 → 03 → 04 → 06 → 08, in that order, plus the `python` and one of `langchain/` `langgraph` tracks from Module 16. That sequence gets you from "what is a gradient" to "I can build and reason about a RAG agent" — everything else deepens or hardens what those five modules establish.